MARS: Enabling Autoregressive Models Multi-Token Generation

Ziqi Jin[1] **Lei Wang**[2] **Ziwei Luo**[3] **Aixin Sun**[1]

[1]Nanyang Technological University [2]Singapore Management University [3]Uppsala University

Abstract

Autoregressive (AR) language models generate text one token at a time, even when consecutive tokens are highly predictable given earlier context. We introduce **MARS** (**M**ask **A**uto**R**egre**S**sion), a lightweight fine-tuning method that teaches an instruction-tuned AR model to predict multiple tokens per forward pass. MARS adds no architectural modifications, no extra parameters, and produces a single model that can still be called exactly like the original AR model with no performance degradation. Unlike speculative decoding, which maintains a separate draft model alongside the target, or multi-head approaches such as Medusa, which attach additional prediction heads, MARS requires only continued training on existing instruction data. When generating one token per forward pass, MARS matches or exceeds the AR baseline on six standard benchmarks. When allowed to accept multiple tokens per step, it maintains baseline-level accuracy while achieving 1.5–1.7× throughput. We further develop a block-level KV caching strategy for batch inference, achieving up to 1.71× wall-clock speedup over AR with KV cache on Qwen2.5-7B. Finally, MARS supports real-time speed adjustment via confidence thresholding: under high request load, the serving system can increase throughput on the fly without swapping models or restarting, providing a practical latency–quality knob for deployment. [1]

1 Introduction

Autoregressive (AR) language models spend the same compute on every token: one forward pass produces exactly one token, whether that token is the inevitable "`the answer is`" following a multiple-choice prompt or a genuinely uncertain next word in an open-ended generation. This uniform cost is wasteful.

Existing approaches to multi-token generation all modify the deployment stack. Speculative decoding [4, 22] maintains a separate draft model alongside the target, doubling memory footprint and adding orchestration complexity. Multi-head approaches such as Medusa [3] and EAGLE [24] attach additional prediction heads to the architecture, requiring extra parameters and head-specific training. Both families introduce components beyond the original model, complicating production serving pipelines.

In this work we take a different angle: rather than building auxiliary components around an AR model, we ask whether lightweight fine-tuning alone can give the model *an additional capability to generate multiple tokens per forward pass*, while preserving its original behavior as a standard AR model. The resulting model should be a strict superset: it can still be served exactly like the original, producing one token at a time with no quality loss, but it also optionally accepts multiple tokens per step when confident, accelerating generation with minimal cost.

[1]Our code is available at: `https://github.com/Xalp/MARS`

Question: Toulouse has twice as many sheep as Charleston. Charleston has 4 times as many sheep as Seattle. How many sheep do Toulouse, Charleston, and Seattle have together if Seattle has 20 sheep?

MARS-7B (confidence threshold τ=0.95), 66 forwards, 2.55 tokens/forward:

Let 's break it down step by step: 1 . Seattle has 20 sheep. 2. Charleston has 4 times as many sheep as Seattle , so Charleston has 4 x 20 = 80 sheep. 3. Toulouse has twice as many sheep as Charleston, so T oulouse has 2 x 80 = 16 0 sheep. Now , let 's add up … Total number of sheep : 20 + 80 + 16 0 = 1 00 + 160 = 260 sheep

Shaded regions = multiple tokens accepted in one forward pass. Unshaded tokens are generated one at a time. Darker = more tokens per step. Formulaic phrases and calculations are chunked; novel reasoning proceeds token-by-token.

Figure 1: Example MARS generation on GSM8K. The model adaptively generates 1–4 tokens per forward pass based on confidence: predictable continuations are batched together, while novel content proceeds token-by-token. This achieves 2.55× token per forward over standard AR decoding.

A natural starting point is block masked diffusion, which trains the model to predict multiple tokens per step within a fixed-size block. However, existing AR-to-block-diffusion conversions result in substantial quality degradation, particularly on reasoning and coding tasks [1, 44, 16, 11]. In Section 3.1, we identify four gaps between AR and block diffusion that may account for this degradation, but find that only one is inherent to multi-token prediction: the use of mask tokens as placeholders for unknown future tokens. The other three arise from design choices that unnecessarily depart from the original AR model, and can be fully closed.

From these motivations, we introduce **MARS** (**M**ask **A**uto**R**egre**S**sion). MARS maintains causal intra-block attention, uses right-shifted logits, and generates strictly left-to-right, leaving only the inherent masking gap. As shown in Figure 1, the resulting model adaptively generates multiple tokens per forward pass when confident, while falling back to single-token generation for novel content. When generating one token per step, MARS matches or exceeds the original AR baseline on six benchmarks, meaning there is *no quality cost* for gaining the option to accelerate. When multi-token generation is enabled, MARS achieves 1.5–1.7× throughput while maintaining baseline-level accuracy.

Contributions.

- We analyze the four gaps between AR and block-masked prediction and show that three of them are eliminable design choices rather than inherent limitations. Closing these gaps is sufficient to recover baseline quality without architectural changes.
- We propose MARS, a lightweight fine-tuning method that teaches an instruction-tuned AR model to optionally predict multiple tokens per forward pass, with no architectural changes, no additional parameters, and reusing the same SFT data. The resulting model can be used exactly like the original AR model with no quality loss, or switched to multi-token mode for faster generation on the fly.
- We identify an auxiliary SFT loss on the clean input stream as the key ingredient for preserving performance at larger block sizes, where the fraction of AR-like training signal would otherwise decay.
- We develop a block-level KV caching strategy for batch inference and demonstrate wall-clock speedups of up to 1.71× over AR with KV cache on Qwen2.5-7B.

2 Background and Related Work

An autoregressive (AR) language model generates text by predicting one token at a time, each conditioned on all previous tokens via causal (left-to-right) attention. Generating T tokens requires T serial forward passes, so inference cost scales linearly with output length regardless of how predictable individual tokens are. If we want a single forward pass to predict multiple future tokens using the same backbone and language model head, a natural construction is *block-masked prediction*: replace a contiguous block of B future tokens with `[MASK]` placeholders and train the model to recover them,

conditioned on the clean tokens from all preceding blocks. To train all blocks in parallel within a single forward pass, we concatenate the clean and masked sequences: $[\mathbf{x}; \tilde{\mathbf{x}}]$, with a structured attention mask that controls visibility (detailed in Section 3.2). However, directly applying this to a pretrained AR model introduces a mismatch: the model was trained under purely causal, fully visible context, but now must predict from partially masked input with potentially different attention patterns and generation order. This mismatch is the source of the quality degradation observed in prior work [1], and motivates the analysis in Section 3.1.

Discrete diffusion and masked language models. Diffusion models for discrete data [2, 23] generate tokens through iterative denoising. Early non-autoregressive work in machine translation [17, 14] demonstrated significant speedups at the cost of quality. More recently, masked diffusion language models [33, 35, 27, 13, 42, 28] apply iterative unmasking to text generation and have begun to close the quality gap with AR models. These models typically use bidirectional attention, a design choice inherited from diffusion models in vision. Several works have converted pretrained AR models into diffusion language models [1, 44, 16, 11], demonstrating that the transition is feasible without training from scratch, though often with significant quality degradation due to the mismatch between the AR model's learned causal attention pattern and the bidirectional attention adopted during conversion.

Causality and multi-token generation. A growing body of work shows that causal attention does not conflict with multi-token parallel generation. Block Diffusion [1] uses causal attention across blocks while keeping bidirectional attention within each block. Building on this, several works further accelerate diffusion LM inference [7, 6], including Fast-dLLM [40] which introduces KV caching for bidirectional diffusion models, and self-speculative decoding [12] which uses the diffusion model itself as both drafter and verifier. CARD [32, 18] find causality actually *improves* over bidirectional alternatives. Some works also suggest that training with a progressive block size [41] or inference with a flexible block size can also gain improvements [38]. ARMD [20] and A3 [9] similarly adopt causal structure for masked diffusion and any-order generation, respectively. In fact, DLM without causality may face a marginal trap [19]. Eso-LM [34] further unifies AR and masked diffusion under causal attention with KV caching. Together, these results establish that bidirectional attention in diffusion language models was a design choice inherited from vision, not a requirement for parallel token generation.

This observation motivates a natural question: if causality and multi-token generation are compatible, how much modification does an AR model actually need to generate multiple tokens at once? Our gap analysis (Section 3.1) suggests the answer is surprisingly little. Of the four gaps between AR and block-masked prediction, three turn out to be eliminable design choices. The sole inherent gap is some positions must predict from incomplete context, which can be addressed with a simple fine-tuning objective.

Multi-token prediction and parallel decoding. Multi-token prediction (MTP) [15] trains AR models with $k-1$ auxiliary heads to predict future tokens in parallel; DeepSeek-V3 [25] deploys this at 671B scale. Medusa [3] and EAGLE [24] add lightweight heads for speculative multi-token proposals. These methods require additional parameters and architectural modifications. MARS achieves multi-token prediction without extra heads, using the same language model head for all positions within a block. Speculative decoding [22, 4] avoids modifying the target model but requires maintaining a separate draft model.

Jacobi decoding [37] and Lookahead decoding [10] achieve parallel generation from a single unmodified model by treating AR generation as a fixed-point iteration. CLLMs [21] improve upon this by training the model to converge faster under Jacobi iteration. These methods initialize future positions with *random tokens* and rely on convergence from incorrect prefixes. MARS takes a complementary approach: instead of random initialization, we use `[MASK]` tokens as explicit placeholders and train the model to predict from this incomplete context directly. This yields substantially higher acceptance rates (1.5× vs 1.07× tokens per forward for Jacobi; Appendix C).

3 Method

The design of MARS is guided by one principle: **the model must remain a fully functional AR model**. Multi-token prediction is an added capability, not a replacement. We begin by analyzing

Table 1: Gaps between AR and block diffusion. MARS aligns with AR on gaps (2)–(4), leaving only the inherent masking gap.

	AR	MARS	Block Diffusion
(1) Token Masking	No masking	Masked within block	Masked within block
(2) Attention Pattern	Causal	Causal	Bidirectional (intra-block)
(3) Logits Alignment	Right-shifted	Right-shifted	Varies
(4) Generation Order	Left-to-right	Left-to-right	Confidence-based

where prior block-masked approaches break this property, then show how MARS closes every eliminable gap at training and inference.

3.1 Where Does Block-Masked Prediction Fail?

Block-masked prediction fails when it departs from the autoregressive model along multiple axes. We identify four gaps between AR and block diffusion (Table 1). Of these, one is inherent to multi-token prediction and cannot be avoided; the other three are eliminable design choices that unnecessarily break AR compatibility.

Gap (1), *token masking*, is inherent: to predict multiple tokens in parallel, future positions must be replaced with `[MASK]` placeholders. This is the fundamental cost of multi-token prediction. The remaining three gaps are eliminable design choices in prior systems that unnecessarily break AR compatibility. Gap (2), *attention pattern*: some block diffusion approaches [1] use bidirectional attention within blocks, but a bidirectional model is no longer a functional AR model; MARS keeps strictly causal attention everywhere. Gap (3), *logits alignment*: AR models predict x_{t+1} from position t (right-shifted logits); changing this convention breaks the output head's AR function, so MARS preserves it. Gap (4), *generation order*: confidence-based diffusion methods unmask tokens out of order within a block, breaking left-to-right generation; MARS always accepts tokens strictly left-to-right. By closing gaps (2)–(4), the resulting model remains a fully functional AR model, with the only difference being that it sees `[MASK]` placeholders within the current block.

3.2 Training

MARS starts from an AR SFT checkpoint trained on the target instruction-following data with standard next-token prediction. Starting from this checkpoint ensures the model has already absorbed the training data distribution, so that MARS training focuses solely on learning the masked prediction paradigm (see Section 4.1 for details).

The high-level idea is simple: we run two copies of the sequence through the model in parallel. The *clean stream* keeps the original tokens intact and trains the model with ordinary AR next-token prediction. The *noisy stream* replaces each block of B tokens with `[MASK]` placeholders and asks the model to predict them, using the clean prefix from earlier blocks as context. Both streams share the same forward pass through a structured attention mask that enforces the correct visibility for each position. This design closes gaps (2) and (3) from Section 3.1 by construction: attention is strictly causal everywhere, and logits are right-shifted to match the AR convention.

Given a response sequence $\mathbf{x} = (x_1, \ldots, x_L)$, we divide it into blocks of size B and replace all tokens in each block with `[MASK]`:

$$\tilde{\mathbf{x}} = (\underbrace{\texttt{[MASK]}, \ldots, \texttt{[MASK]}}_{B}, \underbrace{\texttt{[MASK]}, \ldots, \texttt{[MASK]}}_{B}, \ldots) \tag{1}$$

The model processes a concatenated input $\mathbf{z} = [\mathbf{x}; \tilde{\mathbf{x}}]$ of length $2L$, where the first L positions are the clean stream and the last L are the noisy stream.

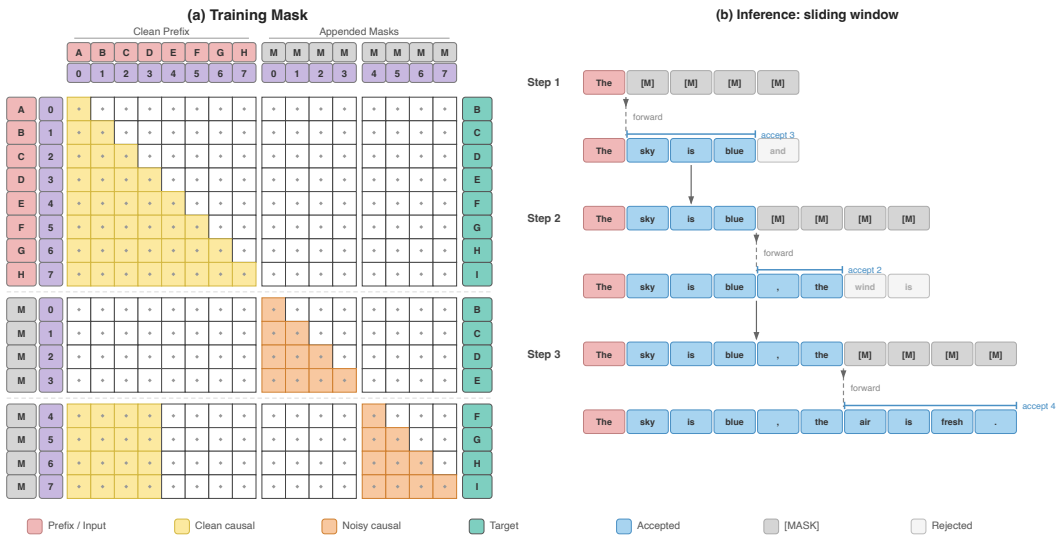

Figure 2: MARS attention mask and inference for L=8, B=4. **Left:** training mask with $[\mathbf{x} \mid \tilde{\mathbf{x}}]$ concatenation. The orange cells show that noisy positions attend to each other *causally* within each block, in contrast to Block Diffusion [1] which uses bidirectional attention within blocks. **Right:** sliding-window inference. The dashed line marks the generation cursor; B `[MASK]` tokens are appended and filled via one forward pass. Accepted tokens (blue) slide into the prefix for the next step.

We define the attention mask $\mathbf{M} \in \{0, -\infty\}^{2L \times 2L}$ over $\mathbf{z}$. Let $\beta(t) = \lceil t/B \rceil$ denote the block index of position t within either stream. For query position i and key position j:

$$M_{ij} = \begin{cases} 0 & \text{if } i, j \le L,\ j \le i \quad \text{(clean causal)} \\ 0 & \text{if } i, j > L,\ \beta(i{-}L) = \beta(j{-}L),\ j \le i \quad \text{(noisy intra-block causal)} \\ 0 & \text{if } i > L,\ j \le L,\ \beta(j) < \beta(i{-}L) \quad \text{(noisy} \to \text{clean cross-stream)} \\ -\infty & \text{otherwise} \end{cases} \tag{2}$$

The *clean causal* case gives the clean stream (positions $1, \ldots, L$) standard causal self-attention, identical to AR training. The *noisy intra-block causal* case allows each noisy position to attend causally within its own block (only `[MASK]` tokens, so only positional information flows). The *cross-stream* case allows each noisy block k to see clean tokens from blocks $1, \ldots, k{-}1$, providing the prefix context needed for prediction.

The training loss on the noisy stream is cross-entropy over masked positions:

$$\mathcal{L}_{\text{mask}} = -\sum_{t \in \mathcal{M}} \log p_\theta(x_t \mid \tilde{\mathbf{x}}, \mathbf{x}_{<\beta(t)}) \tag{3}$$

where $\mathcal{M}$ denotes the set of masked positions and $\mathbf{x}_{<\beta(t)}$ denotes clean tokens from all blocks preceding block $\beta(t)$.

3.3 Preserving Autoregressive Competence

Closing gaps (2)–(4) ensures that MARS *behaves* like an AR model at inference. But training must also ensure the model *remains* an AR model in terms of capability. Block-masked training, by itself, gradually erodes the AR signal, and this erosion gets worse exactly when larger blocks would be most useful. The SFT loss is not a regularization trick; it is the mechanism that keeps the model's AR competence intact while it learns block prediction.

Within a block of size B, position t (1-indexed within the block) is conditioned on $t{-}1$ masked tokens from the same block, plus the fully clean prefix from prior blocks. Only the first position (t=1) sees entirely clean context, making its prediction exactly equivalent to AR next-token prediction. Positions t=2, $\ldots$, B see progressively more `[MASK]` tokens in place of real context, with position

B seeing $B-1$ placeholders. As a simple proxy for how much AR-like signal the model receives, we count the fraction of positions with *fully clean* context:

$$r_{\text{AR}}^{(\text{mask only})} = \frac{1}{B} \tag{4}$$

This is a coarse measure—positions near the start of a block still receive mostly clean context—but it captures the trend: as B grows, the training signal becomes increasingly unlike standard AR. For B=4 the ratio is 25%; for B=8, 12.5%; for B=16, just 6.25%. Empirically, this decay leads to significant degradation on reasoning and coding tasks at larger block sizes (Table 2).

The clean stream in $\mathbf{z} = [\mathbf{x}; \tilde{\mathbf{x}}]$ already uses standard causal attention on the original tokens, and its logits are computed during the forward pass at no extra cost. By adding an AR loss on these logits, we ensure the model never stops practicing next-token prediction, no matter how large the block size:

$$\mathcal{L} = \mathcal{L}_{\text{mask}} + \mathcal{L}_{\text{AR}} \tag{5}$$

where $\mathcal{L}_{\text{AR}} = -\sum_{t=1}^{L} \log p_\theta(x_t \mid \mathbf{x}_{<t})$ is the standard next-token prediction loss computed from clean stream logits. The two terms are weighted equally.

With this combined loss, the AR-equivalent signal fraction becomes:

$$r_{\text{AR}}^{(\text{combined})} = \frac{L + L/B}{2L} = \frac{1 + 1/B}{2} \tag{6}$$

where the numerator counts L pure AR terms from the clean stream plus L/B AR-equivalent terms from the masked stream (one per block). For B=4, this is 62.5%; for B=16, 53.1%. In all cases the ratio stays above 50%, effectively decoupling the AR signal from block size. The model simultaneously learns to predict from masked context *and* maintains its original autoregressive competence, rather than replacing one with the other.

By default MARS includes the SFT loss. To isolate its effect, we also evaluate a variant trained without it (denoted "MARS w/o SFT loss"), which uses $\mathcal{L}_{\text{mask}}$ alone.

3.4 Inference: Left-to-Right Sliding Window

The inference procedure completes the AR-consistent design by closing gap (4): tokens are always accepted strictly left-to-right, matching AR generation order. Combined with causal attention and right-shifted logits from training, the result is that MARS at inference is indistinguishable from AR generation when only one token is accepted per step, and smoothly extends to multi-token generation when the model is confident.

At each step, B number of `[MASK]` tokens are appended after the current prefix and the model runs a single forward pass with pure causal attention to obtain logits for all B positions. Starting from the leftmost masked position, tokens are accepted consecutively while $\max_v p(x_t{=}v) \geq \tau$, with at least one token always accepted. The N accepted tokens join the prefix, and N new `[MASK]` tokens are appended to keep the window at size B. This repeats until generation is complete. The guarantee that at least one token is always accepted ensures that the model degrades gracefully to standard AR decoding when no prediction is confident enough.

The threshold τ directly controls throughput: $\tau \to 1.0$ accepts at most one token per step (recovering exact AR behavior), while lower τ accepts more tokens per step at some quality cost. Crucially, τ can be adjusted *on-the-fly per request* during serving, without retraining or loading a different model. Section 4.4 characterizes the full tradeoff curve.

4 Experiments

We organize experiments around three claims: MARS preserves the original AR model's quality (Section 4.2), the SFT loss is necessary and sufficient for this preservation at scale (Section 4.3), multi-token generation provides a smooth and controllable speed–quality frontier (Section 4.4), and these gains translate to wall-clock speedup in batch inference (Section 4.5).

Table 2: One-token mode (τ=1.0): MARS vs. AR SFT, compute-matched AR SFT (10 epochs), and Block Diffusion. All models generate one token per forward pass. **Bold**: best per column within each scale.

Model	IFEval	BBH	MMLU-Pro	GPQA	GSM8K	HumanEval	Avg
Qwen2.5-0.5B-Instruct							
Base (no fine-tune)	27.8	6.1	2.4	13.6	27.4	26.2	17.2
AR SFT (5 ep)	48.4	26.3	11.9	17.9	32.0	35.4	28.7
AR SFT (10 ep)	47.8	26.3	9.3	14.1	28.3	32.3	26.4
Block Diffusion [1] (B=4)	47.1	7.5	2.0	17.9	30.6	31.7	22.8
MARS-0.5B w/o SFT loss (B=4)	48.5	**27.4**	12.3	19.0	29.5	33.5	28.4
MARS-0.5B (B=4)	**51.3**	26.6	**12.4**	**19.4**	**32.8**	**40.2**	**30.4**
Qwen2.5-7B-Instruct							
Base (no fine-tune)	63.4	24.0	10.5	11.6	57.3	82.9	41.6
AR SFT	67.0	54.0	43.9	**27.5**	68.7	78.7	56.6
MARS-7B (B=4)	**68.2**	**54.6**	**44.4**	26.6	**73.2**	**81.7**	**58.1**

4.1 Setup

We evaluate at two scales: Qwen2.5-0.5B-Instruct and Qwen2.5-7B-Instruct [30], both trained on Dolci-Instruct-SFT [29] ($\sim$2M examples). We first train an AR SFT model for 5 epochs with standard next-token prediction, then continue with MARS training on the same data for another 5 epochs. At 0.5B we train with block sizes $B \in \{4, 8, 16\}$; at 7B we use B=4. Full hyperparameters are in Appendix A.

We compare against the AR SFT starting point and Block Diffusion [1] (0.5B only). Evaluation uses six benchmarks: IFEval (0-shot) [43], BBH (3-shot) [36], MMLU-Pro (0-shot) [39], GPQA (0-shot) [31], GSM8K (0-shot) [8], and HumanEval (0-shot) [5], all with greedy decoding and max 256 new tokens.

4.2 MARS Preserves AR Quality in One-Token Mode

The first claim is that MARS is a strict superset of AR: when generating one token per step, it should match or exceed the original model. Table 2 confirms this at both scales. At 0.5B, MARS achieves 30.4 average versus AR SFT's 28.7, with a 4.8-point gain on HumanEval (40.2 vs 35.4). At 7B, MARS achieves 58.1 versus 56.6, with notable gains on GSM8K (+4.5) and HumanEval (+3.0). The multi-token capability comes at no cost to one-token quality; the additional masked-prediction training appears to act as a form of data augmentation that slightly *improves* the AR mode.

To rule out that the gains simply arise from extra training compute, we include a compute-matched AR SFT baseline trained for 10 epochs (same total fine-tuning budget as MARS: 5 epochs AR + 5 epochs masked). Continuing AR SFT beyond 5 epochs actually *hurts*: the average drops from 28.7 to 26.4, with MMLU-Pro falling from 11.9 to 9.3 and GSM8K from 32.0 to 28.3. The additional AR epochs overfit rather than improve, confirming that MARS's gains come from the masked prediction objective, not from additional training alone.

Block Diffusion [1], which uses bidirectional intra-block attention, tells the opposite story: it collapses on BBH (7.5) and MMLU-Pro (2.0), scores comparable to the untuned base model. This validates the gap analysis from Section 3.1: not all block prediction formulations are compatible with AR pretraining. The ones that break causality, logits alignment, or generation order destroy the model's reasoning ability. MARS avoids all three.

4.3 Why Larger Blocks Work: Validating the Signal Decay Hypothesis

Section 3.3 predicted that without the SFT loss, the AR training signal decays as $1/B$, causing larger blocks to degrade. Table 3 confirms this directly.

Without the SFT loss, increasing block size from 4 to 16 drops the average from 28.4 to 22.2, a loss of 6.2 points. GSM8K falls from 29.5 to 21.0; HumanEval from 33.5 to 20.7. The degradation is

Table 3: Effect of SFT loss across block sizes at both scales (τ=1.0). Without the SFT loss, larger blocks rapidly erode 0.5B quality; with it, performance is stable across B. At 7B (trained only with SFT loss), quality is preserved across all block sizes tested. B=4 rows match Table 2.

Model	IFEval	BBH	MMLU-Pro	GPQA	GSM8K	HumanEval	Avg
Qwen2.5-0.5B-Instruct							
AR SFT	48.4	26.3	11.9	17.9	32.0	35.4	28.7
MARS-0.5B w/o SFT loss							
B=4	48.5	27.4	12.3	19.0	29.5	33.5	28.4
B=8	48.5	24.3	11.1	20.8	24.9	28.7	26.4
B=16	42.6	21.7	10.6	16.3	21.0	20.7	22.2
MARS-0.5B with SFT loss							
B=4	51.3	26.6	12.4	19.4	32.8	40.2	30.4
B=8	49.6	26.9	12.0	19.6	32.8	37.2	29.7
B=16	50.7	27.0	12.1	17.9	33.8	36.6	29.7
Qwen2.5-7B-Instruct							
AR SFT	67.0	54.0	43.9	27.5	68.7	78.7	56.6
MARS-7B with SFT loss							
B=4	68.2	54.6	44.4	26.6	73.2	81.7	58.1
B=8	63.2	55.8	45.4	27.0	71.0	79.9	57.1
B=16	61.9	55.2	44.2	25.7	75.1	76.8	56.5

systematic and concentrated in reasoning and coding tasks, exactly the capabilities most sensitive to the quality of the AR signal.

With the SFT loss, the same block size increase causes a drop of only 0.7 points (30.4 to 29.7). GSM8K actually *improves* from 32.8 to 33.8, and HumanEval drops modestly from 40.2 to 36.6. The SFT loss stabilizes the AR signal ratio above 50% regardless of B (Eq. 6), and the empirical results match this prediction closely. The same pattern holds at 7B: moving from B=4 to B=16 under the SFT loss only drops the average from 58.1 to 56.5 (1.6 points), and all three block sizes remain at or above the AR SFT 7B baseline (56.6).

Figure 3 shows the same pattern across the full threshold sweep: at every operating point (every tokens-per-forward rate), MARS with SFT loss achieves higher accuracy than without, and the gap widens at larger block sizes. The SFT loss does not just help at τ=1.0; it lifts the entire Pareto frontier.

4.4 A Smooth Speed–Quality Frontier via Confidence Thresholding

Having established that MARS preserves AR quality, we now show that the same model provides a smooth, controllable tradeoff when multi-token generation is enabled. Table 4 reports accuracy at τ=0.95 alongside the change from one-token mode. Full threshold sweep results are in Table 8 (Appendix).

The accuracy cost of multi-token generation is small and predictable. At 0.5B with B=8, the average drops by just 1.1 points (29.7→28.6) while generating 1.49 tokens per forward pass. Most individual benchmarks lose less than 1 point. The largest drops are on IFEval (∼5pp): IFEval evaluates strict adherence to formatting instructions (e.g., "write exactly 3 paragraphs"), and multi-token acceptance tends to skip over format-critical tokens that the model would have produced more carefully in single-token mode. At 7B, the tradeoff is even more favorable across all tested block sizes: B=4 loses 1.3 points at 1.68 tok/fwd; B=8 loses 1.0 points at 1.82 tok/fwd, reaching 3.22 on BBH; B=16 loses 1.4 points at 1.87 tok/fwd, reaching 3.29 on BBH. Larger blocks trade a modest additional accuracy cost for higher tokens-per-forward, but in all cases the drop is concentrated in IFEval and HumanEval and never exceeds a few points.

Crucially, the 7B model at τ=0.95 (56.8) still exceeds the AR SFT baseline (56.6). The frontier is smooth with no cliff where quality suddenly collapses, giving the serving system fine-grained control.

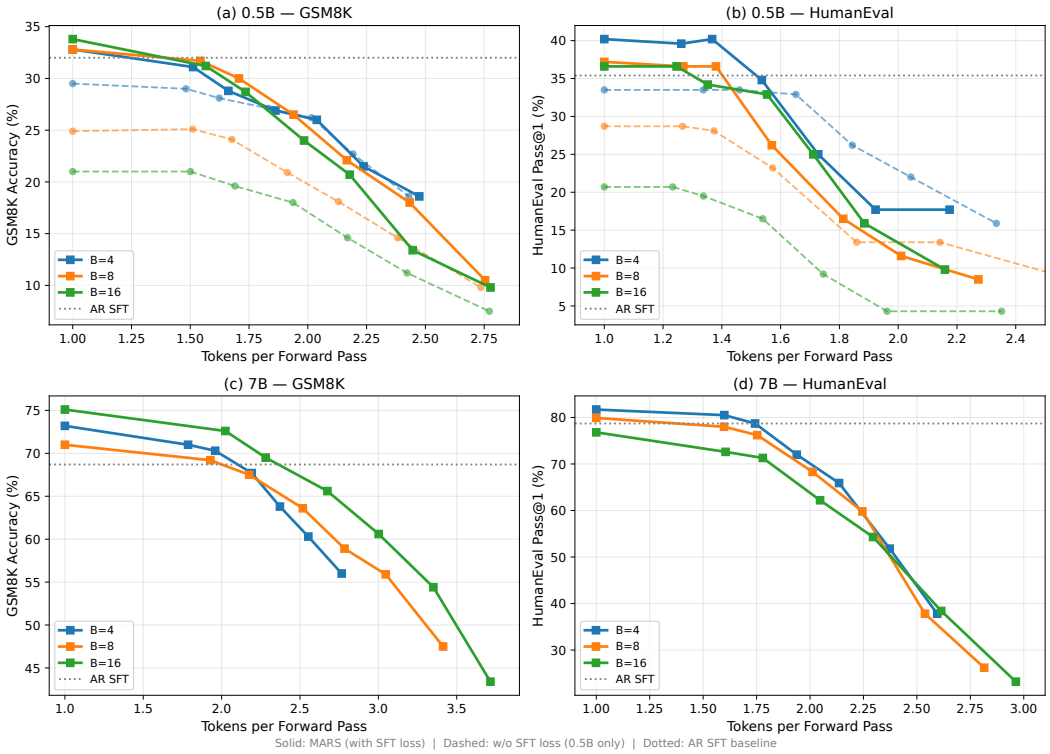

Figure 3: Speed–quality Pareto curves on GSM8K (left) and HumanEval (right). Solid lines: MARS (with SFT loss). Dashed lines: w/o SFT loss. Dotted: AR SFT baseline. With SFT loss, MARS dominates at every operating point on both tasks.

This is the "opt-in" property promised in Section 1: the same checkpoint serves quality-sensitive requests at τ=1.0 and latency-sensitive requests at lower τ, with no model swap and no retraining.

4.5 Wall-Clock Speedup with Block-Level KV Cache

Tokens per forward pass measures algorithmic speedup, but wall-clock throughput is what matters in production. Standard AR decoding benefits from KV cache: each step processes only one new token. MARS processes B masked tokens per step, so without caching, full-sequence recomputation makes it *slower* than AR at batch sizes above 1.

We implement a block-level KV cache strategy (Figure 4): (1) compute the prefix KV cache once per block via a full forward pass, (2) iterate within the block using the cached prefix, where each inner step only forwards B tokens, (3) once all samples in the batch have filled the block, extend the cache with the completed block and advance to the next. Faster samples idle at block boundaries until the slowest sample finishes. Table 5 compares three configurations on GSM8K (256 questions) with Qwen2.5-7B at τ=0.95.

Block-level KV caching is essential for MARS: without it, throughput actually *decreases* as batch size grows (127→112→98 tok/s) due to $O(T^2)$ full-sequence recomputation per step. With the cache, MARS outperforms AR at every batch size tested. At Batch size=4, the best configuration (B_{cache}=32) finishes in 161.2s versus AR's 276.2s, a **1.71**× wall-clock speedup. At Batch size=8, MARS achieves **1.60**× (105.6s vs 169.1s). At Batch size=16, MARS achieves **1.34**× (68.7s vs 91.8s). The speedup is largest at smaller batch sizes where AR's per-token overhead is proportionally higher. The optimal cache granularity shifts with batch size (B_{cache}=32 at Batch size=4/8, B_{cache}=16 at Batch size=16), reflecting a tradeoff between amortizing prefix recomputation and synchronization overhead at block boundaries. Accuracy is preserved across all configurations (78–82% GSM8K, within 2–3pp of AR).

Table 4: Multi-token mode. Each cell shows accuracy with the absolute change from τ=1.0 to τ=0.95 in parentheses. Gray italic rows show tokens accepted per forward pass for each task. Full threshold sweep in Appendix Table 8.

Model	IFEval	BBH	MMLU-Pro	GPQA	GSM8K	HumanEval	Avg
Qwen2.5-0.5B-Instruct							
MARS B=4	45.5 (−5.8)	26.6 (0.0)	11.5 (−0.9)	19.9 (+0.5)	31.1 (−1.7)	39.6 (−0.6)	29.0 (−1.4)
tok/fwd	*1.20*	*1.90*	*1.50*	*1.36*	*1.51*	*1.26*	*1.46*
MARS B=8	44.0 (−5.6)	28.3 (+1.4)	11.9 (−0.1)	19.0 (−0.6)	31.7 (−1.1)	36.6 (−0.6)	28.6 (−1.1)
tok/fwd	*1.23*	*1.96*	*1.49*	*1.46*	*1.54*	*1.27*	*1.49*
MARS B=16	45.3 (−5.4)	25.9 (−1.1)	11.9 (−0.2)	17.9 (0.0)	31.2 (−2.6)	36.6 (0.0)	28.1 (−1.6)
tok/fwd	*1.21*	*1.86*	*1.51*	*1.43*	*1.57*	*1.25*	*1.47*
Qwen2.5-7B-Instruct							
MARS B=4	63.0 (−5.2)	54.3 (−0.3)	44.2 (−0.2)	27.7 (+1.1)	71.0 (−2.2)	80.5 (−1.2)	56.8 (−1.3)
tok/fwd	*1.15*	*2.60*	*1.47*	*1.47*	*1.79*	*1.60*	*1.68*
MARS B=8	62.5 (−0.7)	55.0 (−0.8)	45.1 (−0.3)	26.6 (−0.4)	69.2 (−1.8)	78.0 (−1.9)	56.1 (−1.0)
tok/fwd	*1.16*	*3.22*	*1.54*	*1.49*	*1.93*	*1.60*	*1.82*
MARS B=16	61.7 (−0.2)	53.8 (−1.4)	43.9 (−0.3)	25.9 (+0.2)	72.6 (−2.5)	72.6 (−4.2)	55.1 (−1.4)
tok/fwd	*1.16*	*3.29*	*1.63*	*1.54*	*2.02*	*1.60*	*1.87*

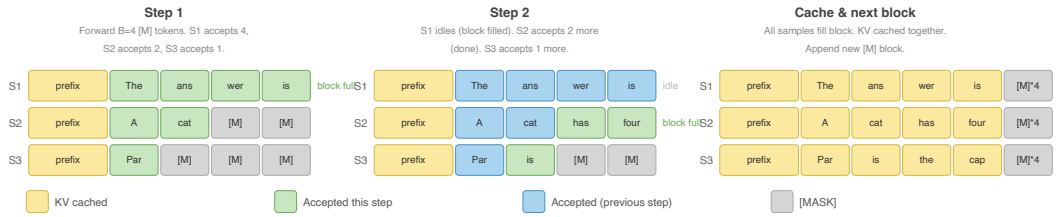

Figure 4: Block-level KV cache for batch inference (B_{cache}=4, batch size 3). Each step forwards B `[MASK]` tokens against the cached prefix. The cache advances by the minimum number of tokens accepted across all samples: after Step 1 (S1 accepts 4, S2 accepts 2, S3 accepts 1), one token is cached (green). S1 idles while S2 and S3 continue. Once all samples fill the block, the entire block is cached (yellow) and new `[MASK]` tokens are appended for the next block.

4.6 Composing MARS with Speculative Decoding

The multi-token capability of MARS is orthogonal to speculative decoding [22, 4]: the former accelerates each forward pass by generating multiple tokens, the latter reduces the number of expensive target-model forwards by amortizing them across cheap draft-model proposals. We show the two compose naturally for further acceleration.

Setup. In standard speculative decoding, a small draft model proposes K tokens and the target model verifies all K in a single forward pass; any prefix that agrees with the target's distribution is accepted, plus one bonus token at the rejection point. We observe that this single bonus token is an unnecessary bottleneck when the target model is MARS-trained: instead of generating one token after the acceptance point, it can generate up to B additional tokens via confidence-based acceptance.

Concretely, inspired by TiDAR [26], we append K+1 groups of B mask tokens to the verification input, one group for each possible acceptance outcome $j \in \{0, \ldots, K\}$. A custom attention mask ensures that group j can see the prompt plus the first j draft tokens, and is causal within the group, while remaining invisible to the draft tokens and all other groups (Figure 5). Position IDs for group j continue from position j after the prompt. After a single target forward, standard rejection sampling determines the acceptance count j; we then take group j's predictions and apply the same confidence threshold τ to accept additional tokens left-to-right. The total overhead is $(K+1) \times B$ extra positions in one forward pass, negligible relative to the prompt length.

We test three configurations on GSM8K (256 questions), using Qwen2.5-0.5B as draft and Qwen2.5-7B as target, all with K=4, B=4, and τ=0.95.

Table 5: Batch inference on GSM8K (256 questions) with Qwen2.5-7B at τ=0.95. B_{cache}: KV cache synchronization granularity. For each batch size we report tokens/sec, total wall-clock time (seconds), and speedup relative to AR with KV cache (×). **Bold**: best MARS configuration per batch size.

Method	B_{cache}	**Batch size=4**			**Batch size=8**			**Batch size=16**		
		tok/s	time	×	tok/s	time	×	tok/s	time	×
AR (KV cache)	–	143.4	276.2	1.00	236.7	169.1	1.00	434.3	91.8	1.00
MARS (no cache)	–	127.1	306.5	0.90	112.4	346.9	0.49	97.7	399.1	0.23
MARS + block cache	4	157.2	248.8	1.11	236.4	165.4	1.02	399.3	97.2	0.94
MARS + block cache	8	196.0	199.4	1.39	289.9	134.4	1.26	484.1	80.1	1.15
MARS + block cache	16	228.5	170.5	1.62	338.0	115.1	1.47	**566.0**	**68.7**	**1.34**
MARS + block cache	32	**241.9**	**161.2**	**1.71**	**368.4**	**105.6**	**1.60**	544.9	71.5	1.28
MARS + block cache	64	239.5	162.0	1.70	342.6	113.8	1.49	383.8	101.5	0.90
MARS + block cache	128	194.2	200.7	1.38	217.9	178.8	0.95	223.9	174.0	0.53
MARS + block cache	256	122.7	317.6	0.87	128.7	302.7	0.56	124.1	313.9	0.29

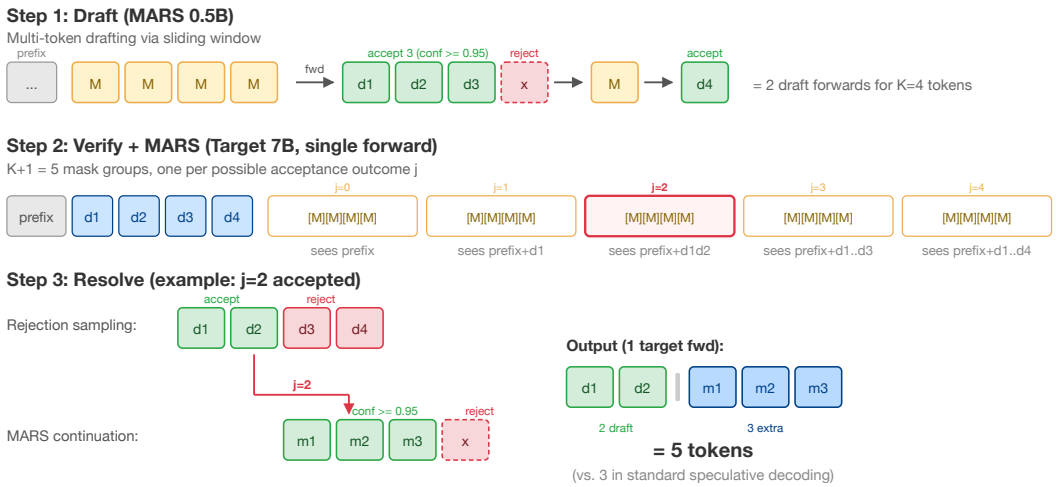

Figure 5: MARS-enhanced speculative decoding (K=4, B=4). Step 1: the MARS draft model proposes K tokens via multi-token sliding window (2 forwards instead of 4). Step 2: the target model verifies draft tokens and simultaneously predicts B continuation tokens for each possible acceptance outcome j, all in a single forward. Step 3: rejection sampling selects j=2; the corresponding group yields 3 additional MARS tokens at τ=0.95, for a total of 5 tokens from one target forward (vs. 3 in standard speculative decoding).

Table 6 shows that MARS provides additive gains at both ends of the pipeline. Replacing the AR target with a MARS target increases tokens per target forward from 3.56 to 4.14, yielding a 1.27× wall-clock speedup. Using MARS for drafting as well reduces draft forwards by 24% (18,959 vs. 24,857) since each draft forward produces multiple tokens, pushing the overall speedup to **1.40**× over standard speculative decoding. The two sources of acceleration are complementary: MARS-enhanced verification produces more tokens per target forward, while MARS drafting reduces the number of (cheap but nonzero) draft forwards. Crucially, accuracy is preserved: the Δ column shows that all configurations match or slightly exceed the corresponding target model's one-token mode accuracy on GSM8K (Table 2).

5 Conclusion

We presented MARS, a lightweight fine-tuning method that gives instruction-tuned AR models the ability to generate multiple tokens per forward pass, with no architectural changes, no additional parameters, and a single checkpoint. The resulting model is a strict superset of the original: in one-token mode it matches or exceeds AR SFT at both 0.5B (+1.7 avg) and 7B (+1.5 avg), and in multi-token mode it achieves 1.5–1.7× throughput with minimal accuracy cost. With block-level

Table 6: Speculative decoding on GSM8K (256 questions) with Qwen2.5-0.5B draft and Qwen2.5-7B target, K=4, τ=0.95. *GSM8K* Δ: accuracy change relative to the target model's one-token mode (Table 2). *tok/target*: average tokens produced per target forward. *Extra/target*: average continuation tokens per target forward beyond accepted drafts (1.00 in standard speculative decoding).

Draft	**Target**	Δ	**tok/s**	**Time (s)**	**Speedup**	**tok/target**
AR SFT (0.5B)	AR SFT (7B)	+0.1	38.1	582	1.00×	3.56
AR SFT (0.5B)	MARS (7B, B=4)	+1.4	53.3	457	1.27×	**4.14**
MARS (0.5B, B=4)	MARS (7B, B=4)	+1.0	**58.6**	**417**	**1.40**×	4.07

KV caching, these gains translate to up to 1.71× wall-clock speedup over AR with KV cache on Qwen2.5-7B. The multi-token capability further composes with speculative decoding, providing a 1.40× speedup over standard speculative decoding when applied to both draft and target models.

The method rests on two insights. First, block-masked prediction fails when it unnecessarily departs from AR behavior; closing the three eliminable gaps (attention pattern, logits alignment, generation order) is sufficient to recover baseline quality. Second, the SFT loss on the clean stream preserves the model's AR competence during masked-prediction training, preventing the AR signal from decaying as $1/B$ and stabilizing it above 50% regardless of block size.

Due to computation constraints, we evaluate only B=4 for the 7B model. Our 0.5B experiments show that different block sizes yield similar Pareto frontiers on the speed–quality tradeoff (Table 3), with B=4 marginally ahead (less than 0.5pp in average accuracy). We expect this pattern to hold at larger scales, but verifying this remains future work. Other promising directions include: (1) cursor-based cache management that eliminates block-boundary synchronization, (2) adaptive block size selection based on input complexity, and (3) scaling to longer draft windows and larger models.

Limitations. MARS training concatenates a clean and noisy copy of each sequence, doubling the per-sample sequence length and therefore the training-time compute relative to standard SFT. This overhead is nonetheless lightweight compared to continual pretraining: training requires only 5 epochs of SFT data rather than large-scale pre-training corpora. The speed–quality tradeoff at aggressive thresholds ($\tau < 0.7$) shows substantial quality loss, suggesting room for improvement in the acceptance strategy. The block-level KV cache requires batch synchronization at block boundaries, which limits throughput gains at large batch sizes.

References

[1] Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *The Thirteenth International Conference on Learning Representations*, 2025.

[2] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34:17981–17993, 2021.

[3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, pages 5209–5235. PMLR, 2024.

[4] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[6] Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. Dmax: Aggressive parallel decoding for dllms, 2026.

[7] Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. dparallel: Learnable parallel decoding for dLLMs. In *The Fourteenth International Conference on Learning Representations*, 2026.

[8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[9] Tianqi Du, Lizhe Fang, Weijie Yang, Chenheng Zhang, Zeming Wei, Yifei Wang, and Yisen Wang. Autoregressive models rival diffusion models at any-order generation. *arXiv preprint arXiv:2601.13228*, 2026.

[10] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of llm inference using lookahead decoding. In *International Conference on Machine Learning*, pages 14060–14079. PMLR, 2024.

[11] Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, Jingyu Liu, Chengyue Wu, Hao Zhang, Enze Xie, Song Han, et al. Efficient-dlm: From autoregressive to diffusion language models, and beyond in speed. *arXiv preprint arXiv:2512.14067*, 2025.

[12] Yifeng Gao, Ziang Ji, Yuxuan Wang, Biqing Qi, Hanlin Xu, and Linfeng Zhang. Self speculative decoding for diffusion large language models. *arXiv preprint arXiv:2510.04147*, 2025.

[13] Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky TQ Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. *Advances in Neural Information Processing Systems*, 37:133345–133385, 2024.

[14] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 6112–6121, 2019.

[15] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *International Conference on Machine Learning*, pages 15706–15734. PMLR, 2024.

[16] Shansan Gong, Shivam Agarwal, Yizhe Zhang, Jiacheng Ye, Lin Zheng, Mukai Li, Chenxin An, Peilin Zhao, Wei Bi, Jiawei Han, et al. Scaling diffusion language models via adaptation from autoregressive models. In *The Thirteenth International Conference on Learning Representations*, 2025.

[17] Jiatao Gu, James Bradbury, Caiming Xiong, Victor OK Li, and Richard Socher. Non-autoregressive neural machine translation. In *International Conference on Learning Representations*, 2018.

[18] Michael Hersche, Nicolas Menet, Ronan Tanios, and Abbas Rahimi. Locally coherent parallel decoding in diffusion language models, 2026.

[19] Ziqi Jin, Bin Wang, Xiang Lin, Lidong Bing, and Aixin Sun. On the role of discreteness in diffusion llms, 2025.

[20] Mahdi Karami and Ali Ghodsi. Auto-regressive masked diffusion models. *arXiv preprint arXiv:2601.16971*, 2026.

[21] Siqi Kou, Lanxiang Hu, Zhezhi He, Zhijie Deng, and Hao Zhang. Cllms: Consistency large language models. In *Forty-first International Conference on Machine Learning*, 2024.

[22] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[23] Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. *Advances in neural information processing systems*, 35:4328–4343, 2022.

[24] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning*, pages 28935–28948. PMLR, 2024.

[25] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[26] Jingyu Liu, Xin Dong, Zhifan Ye, Rishabh Mehta, Yonggan Fu, Vartika Singh, Jan Kautz, Ce Zhang, and Pavlo Molchanov. Tidar: Think in diffusion, talk in autoregression. *arXiv preprint arXiv:2511.08923*, 2025.

[27] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning*, pages 32819–32848. PMLR, 2024.

[28] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, JUN ZHOU, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[29] Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, Robert Berry, Saumya Malik, Saurabh Shah, Scott Geng, Shane Arora, Shashank Gupta, Taira Anderson, Teng Xiao, Tyler Murray, Tyler Romero, Victoria Graf, Akari Asai, Akshita Bhagia, Alexander Wettig, Alisa Liu, Aman Rangapur, Chloe Anastasiades, Costa Huang, Dustin Schwenk, Harsh Trivedi, Ian Magnusson, Jaron Lochner, Jiacheng Liu, Lester James V. Miranda, Maarten Sap, Malia Morgan, Michael Schmitz, Michal Guerquin, Michael Wilson, Regan Huff, Ronan Le Bras, Rui Xin, Rulin Shao, Sam Skjonsberg, Shannon Zejiang Shen, Shuyue Stella Li, Tucker Wilde, Valentina Pyatkin, Will Merrill, Yapei Chang, Yuling Gu, Zhiyuan Zeng, Ashish Sabharwal, Luke Zettlemoyer, Pang Wei Koh, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. Olmo 3, 2025.

[30] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.

[31] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023.

[32] Junhao Ruan, Bei Li, Yongjing Yin, Pengcheng Huang, Xin Chen, Jingang Wang, Xunliang Cai, Tong Xiao, and JingBo Zhu. Causal autoregressive diffusion language model. *arXiv preprint arXiv:2601.22031*, 2026.

[33] Subham S Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.

[34] Subham Sekhar Sahoo, Zhihan Yang, Yash Akhauri, Johnna Liu, Deepansha Singh, Zhoujun Cheng, Zhengzhong Liu, Eric Xing, John Thickstun, and Arash Vahdat. Esoteric language models: Bridging autoregressive and masked diffusion llms, 2026.

[35] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. *Advances in neural information processing systems*, 37:103131–103167, 2024.

[36] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them, 2022.

[37] Yao Teng, Han Shi, Xian Liu, Xuefei Ning, Guohao Dai, Yu Wang, Zhenguo Li, and Xihui Liu. Accelerating auto-regressive text-to-image generation with training-free speculative jacobi decoding. In *The Thirteenth International Conference on Learning Representations*, 2024.

[38] Lipeng Wan, Junjie Ma, Jianhui Gu, Zeyang Liu, Xuyang Lu, and Xuguang Lan. Geoblock: Inferring block granularity from dependency geometry in diffusion language models. *arXiv preprint arXiv:2603.26675*, 2026.

[39] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024.

[40] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.

[41] Hanchen Xia, Baoyou Chen, Yutang Ge, Guojiang Zhao, and Siyu Zhu. T*: Progressive block scaling for mdm through trajectory aware rl. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026.

[42] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

[43] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

[44] Zhanhui Zhou, Lingjie Chen, Hanghang Tong, and Dawn Song. dllm: Simple diffusion language modeling, 2026.

A Training Details

Training cost. MARS training concatenates a clean and noisy copy of each sequence, doubling the effective sequence length. This incurs additional training cost: for the 0.5B model, AR SFT takes 15 H200-hours while MARS takes 33 H200-hours (2.2×); for the 7B model, 100 vs 202 H200-hours (2.0×). Peak GPU memory usage is approximately 1.5× that of AR SFT. This overhead is modest compared to continual pretraining: both stages use only 5 epochs of SFT data.

B Threshold Sweep Details

Table 8 reports the full speed–quality tradeoff for MARS across block sizes $B \in \{4, 8, 16\}$ at both 0.5B (with and without SFT loss) and 7B (with SFT loss) on GSM8K and HumanEval, sweeping the acceptance threshold τ from 1.0 (one token per step, equivalent to AR) down to 0.5. At 0.5B, τ=0.95 with B=4 gives 1.51 tokens per forward on GSM8K with only 1.7pp accuracy loss relative to τ=1.0; the SFT loss variant consistently dominates the w/o SFT loss variant across all operating points, confirming that the SFT loss stabilizes performance across the entire Pareto frontier.

At 7B, the frontier moves up sharply across all block sizes: on GSM8K, B=16 reaches 2.02 tok/fwd at τ=0.95 with only a 2.5pp drop from τ=1.0 (75.1→72.6). Scale and block size compound—the 7B model remains confident over longer spans, so larger B translates directly into higher tok/fwd with modest quality cost in the $\tau \geq 0.9$ regime. HumanEval degrades faster at aggressive τ: at B=16, τ=0.6 already drops pass@1 below 40%, reflecting code's strict correctness requirement—format-critical tokens committed prematurely cannot be undone.

Table 7: Training hyperparameters. Both stages (AR SFT → MARS) use identical settings per model size. Effective batch sizes are matched across scales.

Hyperparameter	**0.5B**	**7B**
Base model	Qwen2.5-0.5B-Instruct	Qwen2.5-7B-Instruct
Training data	Dolci-Instruct-SFT (∼2M examples)	
Optimizer	AdamW	
Learning rate	5×10^{-6}	
LR schedule	Cosine decay	
Warmup ratio	0.03	
Epochs	5 (per stage)	
Max sequence length	512	
Per-device batch size	48	24 (AR) / 12 (MARS)
Gradient accumulation	1	2 (AR) / 4 (MARS)
Effective batch size	384	384
Precision	bfloat16	
Hardware	8× NVIDIA H200	
Block sizes tested	4, 8, 16	4
SFT loss	Included (MARS) / Excluded (MARS w/o SFT loss)	
Evaluation		
Generation length	256 tokens	
Decoding	Greedy (temperature = 0)	
Steps (MARS)	256 (one token per step at τ=1.0)	

C Jacobi Decoding Baseline

Table 9 compares Jacobi decoding with MARS on the same AR SFT checkpoint. Jacobi achieves only 1.07× average speedup (tokens per forward), compared to 1.46× for MARS. The limited speedup is expected: in Jacobi, each position sees random tokens as context for earlier positions in the generation region. Since the AR model was never trained to predict from `[MASK]` placeholders, yielding substantially higher acceptance rates.

Jacobi does have one structural advantage: because it initializes all N output positions at once, the model knows the exact generation length from the start, preventing it from generating beyond the intended boundary. This likely explains why Jacobi scores higher than AR SFT on GSM8K (36.5 vs 32.0) and HumanEval (42.1 vs 35.4), where output length control matters for correctness. On format-sensitive and reasoning tasks where this advantage does not apply, Jacobi drops significantly: IFEval (−7.4) and BBH (−7.1).

D Acceptance Metric Sensitivity

The sliding-window inference by default accepts tokens left-to-right while a confidence score exceeds a threshold τ. In the main experiments we use the probability of the top token, $\max_v p(v \mid \cdot)$, as the confidence score. Here we evaluate two alternatives on GSM8K with MARS (B=4):

- **Entropy**: $H(p) = -\sum_v p_v \log p_v$. Accept while $H \leq \tau$. Lower entropy indicates higher confidence.
- **Top-2 margin**: $p_{\text{top}_1} - p_{\text{top}_2}$. Accept while margin $\geq \tau$. A large gap between the best and second-best token indicates high confidence.

Figure 6 shows the speed–quality frontier for each metric. All three trace similar Pareto curves, confirming that the sliding-window acceptance mechanism is robust to the specific confidence measure. Entropy and top-2 margin show marginally smoother degradation at comparable speedups, but the differences are small. We use probability in the main paper for its simplicity.

Table 8: Speed–quality tradeoff on GSM8K (0-shot) and HumanEval (0-shot) at both scales. Tok/Fwd: average tokens accepted per forward pass. τ=1.0: one token per step (from Table 2). Lower τ accepts more tokens but may reduce accuracy.

	GSM8K						**HumanEval**					
	B=4		B=8		B=16		B=4		B=8		B=16	
τ	Acc	T/F	Acc	T/F	Acc	T/F	P@1	T/F	P@1	T/F	P@1	T/F
MARS-0.5B												
AR SFT	32.0	1.00	32.0	1.00	32.0	1.00	35.4	1.00	35.4	1.00	35.4	1.00
1.0	32.8	1.00	32.8	1.00	33.8	1.00	40.2	1.00	37.2	1.00	36.6	1.00
0.95	31.1	1.51	31.7	1.54	31.2	1.57	39.6	1.26	36.6	1.27	36.6	1.25
0.9	28.8	1.66	30.0	1.71	28.7	1.74	40.2	1.37	36.6	1.38	34.2	1.35
0.8	26.9	1.86	26.5	1.94	24.0	1.98	34.8	1.54	26.2	1.57	32.9	1.55
0.7	26.0	2.04	22.1	2.17	20.7	2.18	25.0	1.73	16.5	1.81	25.0	1.71
0.6	21.5	2.24	18.0	2.43	13.4	2.45	17.7	1.92	11.6	2.01	15.9	1.89
0.5	18.6	2.47	10.5	2.75	9.8	2.78	17.7	2.18	8.5	2.27	9.8	2.16
MARS-0.5B w/o SFT loss												
1.0	29.5	1.00	24.9	1.00	21.0	1.00	33.5	1.00	28.7	1.00	20.7	1.00
0.95	29.0	1.48	25.1	1.51	21.0	1.50	33.5	1.34	28.7	1.27	20.7	1.23
0.9	28.1	1.62	24.1	1.68	19.6	1.69	33.5	1.46	28.1	1.37	19.5	1.34
0.8	27.1	1.83	20.9	1.91	18.0	1.94	32.9	1.65	23.2	1.57	16.5	1.54
0.7	26.2	2.02	18.1	2.13	14.6	2.17	26.2	1.84	13.4	1.86	9.2	1.75
0.6	22.7	2.19	14.6	2.38	11.2	2.42	22.0	2.04	13.4	2.14	4.3	1.96
0.5	18.6	2.43	9.8	2.74	7.5	2.77	15.9	2.33	9.2	2.53	4.3	2.35
MARS-7B (with SFT loss)												
AR SFT	68.7	1.00	68.7	1.00	68.7	1.00	78.7	1.00	78.7	1.00	78.7	1.00
1.0	73.2	1.00	71.0	1.00	75.1	1.00	81.7	1.00	79.9	1.00	76.8	1.00
0.95	71.0	1.79	69.2	1.93	72.6	2.02	80.5	1.60	78.0	1.60	72.6	1.60
0.9	70.3	1.96	67.5	2.18	69.5	2.28	78.7	1.74	76.2	1.75	71.3	1.78
0.8	67.7	2.19	63.6	2.52	65.6	2.68	72.0	1.94	68.3	2.01	62.2	2.05
0.7	63.8	2.37	58.9	2.78	60.6	3.00	65.9	2.14	59.8	2.24	54.3	2.29
0.6	60.3	2.55	55.9	3.05	54.4	3.35	51.8	2.37	37.8	2.54	38.4	2.61
0.5	56.0	2.77	47.5	3.41	43.4	3.72	37.8	2.60	26.2	2.82	23.2	2.96

Table 9: Jacobi decoding [37] on the AR SFT checkpoint (0.5B, training-free). Jacobi uses fixed-point iteration: all future positions are initialized with random tokens and iteratively updated via causal forward passes until convergence. Tok/fwd: average tokens accepted per forward pass.

Method	**IFEval**	**BBH**	**MMLU-Pro**	**GPQA**	**GSM8K**	**HumanEval**	**Avg**	**Tok/fwd**
AR SFT	48.4	26.3	11.9	17.9	32.0	35.4	28.7	1.00
Jacobi (on AR SFT)	41.0	19.2	11.7	17.6	36.5	42.1	28.0	1.07
MARS (τ=0.95, B=4)	45.5	26.6	11.5	19.9	31.1	39.6	29.0	1.46

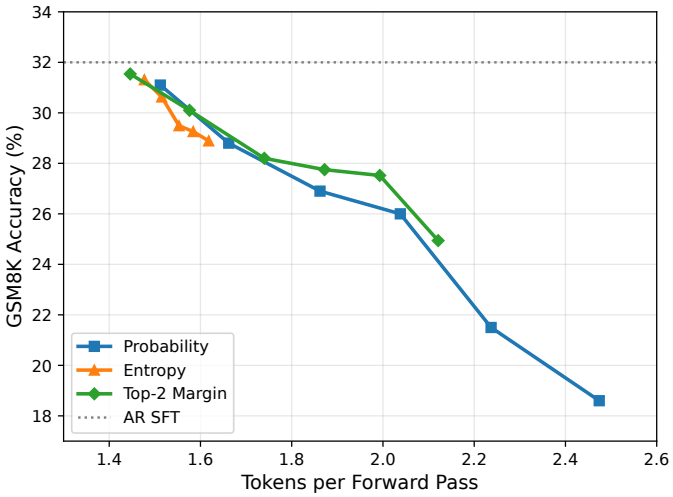

Figure 6: Speed–quality trade-off under three acceptance metrics (MARS $B{=}4$, GSM8K). All three metrics trace similar Pareto frontiers, indicating that the speed–quality trade-off is robust to the choice of acceptance criterion. Entropy and top-2 margin degrade slightly more gracefully than raw probability at comparable tokens per forward pass.